

```

----- archivo firebase.js
// firebase.js - ARCHIVO UNIFICADO
import { initializeApp } from
"https://www.gstatic.com/firebasejs/10.13.0/firebase-app.js";
import {
  getDatabase,
  ref,
  push,
  get,
  update,
  remove,
  query,
  orderByChild
} from "https://www.gstatic.com/firebasejs/10.13.0/firebase-database.js";

const firebaseConfig = {
  apiKey: "AIzaSyAfpEDzd8wc6t9Y3foI2HDrWVL_MIzhYnA",
  authDomain: "asistencia-93328.firebaseio.com",
  databaseURL: "https://asistencia-93328-default-rtdb.firebaseio.com",
  projectId: "asistencia-93328",
  storageBucket: "asistencia-93328.firebaseio.com",
  messagingSenderId: "692275978617",
  appId: "1:692275978617:web:5579ef6c0aeb2d58c7cfa8"
};

// Inicialización única
export const app = initializeApp(firebaseConfig);
export const db = getDatabase(app);

// Nodos / Colecciones
const coleccionCursos = "cursos";
const coleccionPreceptores = "preceptor";
const nodoAlumnos = "tbl_alumnos";
const rutaPreceptores = "preceptor";

/**
 * --- SECCIÓN CURSOS ---
 */
export const addCurso = (curso, ubicacion, capacidad, obs, preceptorID) =>
  push(ref(db, coleccionCursos), { curso, ubicacion, capacidad, obs,
preceptorID });

export const getCursosCollection = () => get(ref(db, coleccionCursos));

export const getCursoCollection = (id) => get(ref(db,
`${coleccionCursos}/${id}`));

export const updateCursoCollection = (id, newFields) =>
  update(ref(db, `${coleccionCursos}/${id}`), newFields);

export const deleteCursoCollection = (id) =>
  remove(ref(db, `${coleccionCursos}/${id}`));

/**

```

```

* --- SECCIÓN PRECEPTORES ---
*/
export const getPreceptoresCollection = () => get(ref(db,
coleccionPreceptores));

export const addNuevopreceptor = (nombre, turno, obs) => {
  // Aquí usamos la ruta como texto
  return push(ref(db, rutaPreceptores), {
    nombre: nombre,
    turno: turno,
    obs: obs
  });
};

export const getPreceptorCollection = (id) => get(ref(db,
`${coleccionPreceptores}/${id}`));

export const updatePreceptorCollection = (id, newFields) =>
  update(ref(db, `${coleccionPreceptores}/${id}`), newFields);

export const deletePreceptorCollection = (id) =>
  remove(ref(db, `${coleccionPreceptores}/${id}`));

/**
* --- SECCIÓN ALUMNOS ---
*/
export const addalumno = (curso, nombre, dni, obs, huellaId) => {
  return push(ref(db, nodoAlumnos), { curso, nombre, dni, obs, huellaId,
borrar: "No" });
};

export const getalumnosCollection = () => {
  const consultaOrdenada = query(ref(db, nodoAlumnos), orderByChild('dni'));
  return get(consultaOrdenada);
};

export const getalumnoCollection = (id) => get(ref(db, `${nodoAlumnos}/${id}`));

export const updatealumnoCollection = (id, newFields) =>
  update(ref(db, `${nodoAlumnos}/${id}`), newFields);

export const deletealumnoCollection = (id) =>
  remove(ref(db, `${nodoAlumnos}/${id}`));

export const marcarParaBorrar = (id) =>
  update(ref(db, `${nodoAlumnos}/${id}`), { obs: "Borrar" });

/**
* --- SECCIÓN ASISTENCIAS ---
*/
export const getAsistenciasHoy = () => get(ref(db, "asistencia"));
----- archivo todocrud.js
-----
// main.js - ARCHIVO UNIFICADO Y LIMPIO

```

```

import {
  addalumno, getalumnosCollection, deletealumnoCollection,
  getalumnoCollection, updatealumnoCollection, marcarParaBorrar,
  addCurso, getCursosCollection, deleteCursoCollection, getCursoCollection,
  updateCursoCollection,
  getPreceptoresCollection
} from "./firebase.js";

/**
 * 1. GESTIÓN DE TABLAS (ALUMNOS Y CURSOS)
 */

async function refrescarTablas() {
  if (document.getElementById("tablaalumnos")) await mostraralumnosEnHTML();
  if (document.getElementById("tablaalumnosSinHuella")) await
mostraralumnosSinHuella();
  if (document.getElementById("tablaCursos")) await mostrarCursosEnHTML();
}

async function mostraralumnosEnHTML() {
  try {
    const tabla = document.querySelector("#tablaalumnos tbody");
    if (!tabla) return;

    const queryCollection = await getalumnosCollection();
    const filtroCurso = document.getElementById("filtroCurso")?.value || "";
    const busqueda =
document.getElementById("buscarAlumno")?.value.toLowerCase() || "";
    const spanContador = document.getElementById("contadorAlumnos");

    tabla.innerHTML = "";
    let contador = 0;

    queryCollection.forEach((doc) => {
      const alumno = doc.val();
      const id = doc.key;

      if (alumno.borrar === "Si") return;
      if (filtroCurso && alumno.curso !== filtroCurso) return;

      const coincideNombre =
alumno.nombre?.toLowerCase().includes(busqueda);
      const coincideDNI = alumno.dni?.toString().includes(busqueda);
      if (busqueda && !coincideNombre && !coincideDNI) return;

      contador++;
      const fila = document.createElement("tr");
      fila.id = id;
      fila.innerHTML = `
        <td>${alumno.curso}</td>
        <td>${alumno.nombre}</td>
        <td>${alumno.dni}</td>
        <td>${alumno.obs}</td>
        <td>

```

```

                <button
onclick="window.miModal('detallealumnoModal','${id}')" class="btn btn-success
btn-sm"><i class="bi bi-binoculars"></i></button>
                <button
onclick="window.miModal('editaralumnoModal','${id}')" class="btn btn-warning
btn-sm"><i class="bi bi-pencil-square"></i></button>
                <button
onclick="window.miModal('eliminaralumnoModal','${id}')" class="btn btn-danger
btn-sm"><i class="bi bi-trash"></i></button>
            </td>`;
            tabla.appendChild(fila);
        });
        if (spanContador) spanContador.textContent = contador;
    } catch (error) { console.error("Error alumnos:", error); }
}

```

```

/**
 * Función para mostrar alumnos que no tienen huella asignada (huellaId === -1)
 */
async function mostraralumnosSinHuella() {
    try {
        const tabla = document.querySelector("#tablaalumnosSinHuella tbody");
        if (!tabla) return; // Si no existe el elemento, salimos sin error

        const queryCollection = await getalumnosCollection();
        tabla.innerHTML = "";

        queryCollection.forEach((doc) => {
            const alumno = doc.val();
            const id = doc.key;

            // Filtros: No borrados Y que no tengan huellaId (valor -1)
            if (alumno.borrar === "Si") return;
            if (alumno.huellaId !== -1 && alumno.huellaId !== "-1") return;

            const fila = document.createElement("tr");
            fila.id = id;
            fila.innerHTML = `
                <td>${alumno.curso}</td>
                <td>${alumno.nombre}</td>
                <td>${alumno.dni}</td>
                <td>${alumno.obs}</td>
                <td>
                    <button
onclick="window.miModal('detallealumnoModal','${id}')" class="btn btn-success
btn-sm">
                        <i class="bi bi-binoculars"></i>
                    </button>
                    <button
onclick="window.miModal('editaralumnoModal','${id}')" class="btn btn-warning
btn-sm">
                        <i class="bi bi-pencil-square"></i>

```

```

        </button>
        <button
onclick="window.miModal('eliminaralumnoModal','${id}')" class="btn btn-danger
btn-sm">
            <i class="bi bi-trash"></i>
        </button>
    </td>`;
    tabla.appendChild(fila);
});
} catch (error) {
    console.error("Error al obtener los alumnos sin huella:", error);
}
}

```

```

async function mostrarCursosEnHTML() {
    try {
        const tabla = document.querySelector("#tablaCursos tbody");
        if (!tabla) return;

        const snapshot = await getCursosCollection();
        tabla.innerHTML = "";

        if (snapshot.exists()) {
            snapshot.forEach((child) => {
                const id = child.key;
                const curso = child.val();
                const fila = document.createElement("tr");
                fila.id = id;
                fila.innerHTML = `
                    <td>${curso.curso || "N/A"}</td>
                    <td>${curso.ubicacion || "N/A"}</td>
                    <td>${curso.capacidad || "N/A"}</td>
                    <td>${curso.obs || "N/A"}</td>
                    <td>
                        <button
onclick="window.miModal('detalleCursoModal','${id}')" class="btn btn-success
btn-sm"><i class="bi bi-binoculars"></i></button>
                        <button
onclick="window.miModal('editarCursoModal','${id}')" class="btn btn-warning
btn-sm"><i class="bi bi-pencil-square"></i></button>
                        <button
onclick="window.miModal('eliminarCursoModal','${id}')" class="btn btn-danger
btn-sm"><i class="bi bi-trash"></i></button>
                    </td>`;
                tabla.appendChild(fila);
            });
        }
    } catch (error) { console.error("Error cursos:", error); }
}

```

```

/**
 * 2. GESTIÓN DE MODALES UNIFICADA

```

```

*/

window.miModal = async function (idModal, idRef = "") {
  try {
    // Limpieza previa
    const existing = document.getElementById(idModal);
    if (existing) {
      const ins = bootstrap.Modal.getInstance(existing);
      if (ins) ins.hide();
      existing.remove();
    }
    document.querySelectorAll('.modal-backdrop').forEach(el => el.remove());

    // Selección de URL
    const rutas = {
      "agregaralumnoModal": "modales/modalAdd.php",
      "detallealumnoModal": "modales/modalDetalles.php",
      "editaralumnoModal": "modales/modalEditar.php",
      "eliminaralumnoModal": "modales/modalDelete.php",
      "agregarCursoModal": "modales/modalAddCurso.php",
      "detalleCursoModal": "modales/modalDetallesCurso.php",
      "editarCursoModal": "modales/modalEditarCurso.php",
      "eliminarCursoModal": "modales/modalDeleteCurso.php"
    };

    if (!rutas[idModal]) return;

    const response = await fetch(rutas[idModal]);
    const text = await response.text();
    const parser = new DOMParser();
    const htmlDoc = parser.parseFromString(text, 'text/html');
    const modalElement = htmlDoc.querySelector('.modal');

    document.body.appendChild(modalElement);
    const myModal = new bootstrap.Modal(modalElement);
    myModal.show();

    // Carga de datos específicos post-apertura
    if (idModal === "agregaralumnoModal") await cargarCursosEnModal();
    if (idModal === "agregarCursoModal") await cargarPreceptoresEnModal();
    if (idModal === "detallealumnoModal") await cargarDetallealumno(idRef);
    if (idModal === "editaralumnoModal") await
getalumnoUpdateCollection(idRef);
    if (idModal === "detalleCursoModal") await cargarDetalleCurso(idRef);
    if (idModal === "editarCursoModal") await
getCursoUpdateCollection(idRef);

    if (idModal.includes("eliminar")) {
      const btn = modalElement.querySelector("#confirmDeleteBtn");
      if (btn) btn.onclick = async () => {
        idModal.includes("alumno") ? await eliminaralumno(idRef) : await
eliminarCurso(idRef);
        myModal.hide();
      };
    }
  }
};

```

```

    }
  } catch (error) { console.error("Error Modal:", error); }
};

/**
 * 3. FUNCIONES DE APOYO (SELECTS Y DETALLES)
 */

async function cargarCursosEnModal() {
  const select = document.getElementById("selectCursoModal");
  if (!select) return;
  const cursosSnap = await getCursosCollection();
  select.innerHTML = '<option selected value="">Seleccione</option>';
  cursosSnap.forEach(doc => {
    const data = doc.val();
    select.innerHTML += `<option
value="${data.curso}">${data.curso}</option>`;
  });
}

async function cargarPreceptoresEnModal() {
  const select = document.getElementById("selectPreceptorModal");
  if (!select) return;
  const snap = await getPreceptoresCollection();
  select.innerHTML = '<option selected value="">Seleccione
Preceptor</option>';
  if (snap.exists()) {
    snap.forEach(doc => {
      select.innerHTML += `<option
value="${doc.key}">${doc.val().nombre}</option>`;
    });
  }
}

/**
 * 4. EVENTOS DE CREACIÓN Y ACTUALIZACIÓN
 */

window.addNuevoCurso = async function (event) {
  event.preventDefault();
  const form = document.querySelector("#formularioCursoEdit");
  const fd = new FormData(form);
  const preceptorID = fd.get("preceptorID");

  if (!preceptorID) return alert("Seleccione un preceptor");

  try {
    await addCurso(fd.get("curso"), fd.get("ubicacion"),
fd.get("capacidad"), fd.get("obsCurso"), preceptorID);

bootstrap.Modal.getInstance(document.getElementById('agregarCursoModal')).hide()
;
    await mostrarCursosEnHTML();
    window.mostrarAlerta({ tipoToast: "success", mensaje: "¡Curso creado!"

```

```

});
    } catch (e) { console.error(e); }
};

window.addNuevoalumno = async function (event) {
    event.preventDefault();
    const fd = new FormData(document.querySelector("#formularioalumno"));
    try {
        await addalumno(fd.get("curso"), fd.get("nombre"), fd.get("dni"),
fd.get("obs"), -1);

bootstrap.Modal.getInstance(document.getElementById("agregaralumnoModal")).hide(
);
        await refrescarTablas();
        window.mostrarAlerta({ tipoToast: "success", mensaje: "¡Alumno
registrado!" });
    } catch (e) { console.error(e); }
};

/**
 * 5. INICIALIZACIÓN GENERAL
 */

window.addEventListener("DOMContentLoaded", async () => {
    await refrescarTablas();

    // Filtros de búsqueda (si existen)
    document.getElementById("buscarAlumno")?.addEventListener("input",
mostraralumnosEnHTML);
    document.getElementById("filtroCurso")?.addEventListener("change",
mostraralumnosEnHTML);

    if (document.getElementById("filtroCurso")) {
        const select = document.getElementById("filtroCurso");
        const cursosSnap = await getCursosCollection();
        cursosSnap.forEach(doc => {
            const data = doc.val();
            select.innerHTML += `<option
value="${data.curso}">${data.curso}</option>`;
        });
    }
});

// Función de Alerta Global
window.mostrarAlerta = function ({ tipoToast, mensaje }) {
    if (typeof iziToast === 'undefined') return alert(mensaje);
    iziToast[tipoToast === "error" ? "error" : "success"]({
        title: tipoToast === "success" ? "¡Éxito!" : "¡Error!",
        message: mensaje,
        position: "topRight"
    });
};
/**
 * READ ONE: Carga los detalles del alumno en el modal

```



```

*/
async function cargarDetallealumno(id) {
  try {
    const alumnoDoc = await getalumnoCollection(id);
    if (alumnoDoc.exists()) {
      const data = alumnoDoc.val();
      const contenedor = document.querySelector("#detallealumnoContenido
ul");
      if (!contenedor) return;

      contenedor.innerHTML = `
        <li class="list-group-item"><b>Curso:</b> ${data.curso ||
"N/A"}</li>
        <li class="list-group-item"><b>Apellido y Nombre:</b>
${data.nombre || "N/A"}</li>
        <li class="list-group-item"><b>DNI:</b> ${data.dni ||
"N/A"}</li>
        <li class="list-group-item"><b>Obs:</b> ${data.obs ||
"N/A"}</li>
      `;
    }
  } catch (error) {
    console.error("Error al cargar detalles alumno:", error);
  }
}

/**
 * READ ONE: Carga los detalles del curso en el modal
 */
async function cargarDetalleCurso(id) {
  try {
    const snapshot = await getCursoCollection(id);
    if (snapshot.exists()) {
      const data = snapshot.val();
      const contenedor = document.querySelector("#detalleCursoContenido
ul");
      if (!contenedor) return;

      contenedor.innerHTML = `
        <li class="list-group-item"><b>Curso: </b> ${data.curso ||
"N/A"}</li>
        <li class="list-group-item"><b>Ubicación:</b> ${data.ubicacion
|| "N/A"}</li>
        <li class="list-group-item"><b>Capacidad:</b> ${data.capacidad
|| "N/A"}</li>
        <li class="list-group-item"><b>Obs: </b> ${data.obs ||
"N/A"}</li>
      `;
    }
  } catch (error) {
    console.error("Error al cargar detalles curso:", error);
  }
}
/**

```

```

* Carga los datos del ALUMNO en el formulario de edición
*/
async function getalumnoUpdateCollection(id) {
  try {
    const alumnoDoc = await getalumnoCollection(id);
    if (alumnoDoc.exists()) {
      const data = alumnoDoc.val();
      // Buscamos por el atributo 'name' para asegurar coincidencia con el
modal
      document.querySelector('[name="idalumno"]').value = id;
      document.querySelector('[name="curso"]').value = data.curso || "";
      document.querySelector('[name="nombre"]').value = data.nombre || "";
      document.querySelector('[name="dni"]').value = data.dni || "";
      document.querySelector('[name="obs"]').value = data.obs || "";
    }
  } catch (error) {
    console.error("Error al cargar datos de alumno para editar:", error);
  }
}

/**
* Carga los datos del CURSO en el formulario de edición
*/
async function getCursoUpdateCollection(id) {
  try {
    const cursoDoc = await getCursoCollection(id);
    if (cursoDoc.exists()) {
      const data = cursoDoc.val();
      // Buscamos por ID o Name según lo tengas en tu modalEditarCurso.php
      if (document.querySelector("#idCurso"))
document.querySelector("#idCurso").value = id;
      if (document.querySelector("#curso"))
document.querySelector("#curso").value = data.curso || "";
      if (document.querySelector("#ubicacion"))
document.querySelector("#ubicacion").value = data.ubicacion || "";
      if (document.querySelector("#capacidad"))
document.querySelector("#capacidad").value = data.capacidad || "";
      if (document.querySelector("#obsCurso"))
document.querySelector("#obsCurso").value = data.obs || "";
    }
  } catch (error) {
    console.error("Error al cargar datos de curso para editar:", error);
  }
}

/**
* Procesa la actualización del ALUMNO
*/
window.actualizaralumno = async function (event) {
  event.preventDefault();
  const formulario = document.querySelector("#formularioalumnoEdit");
  const formData = new FormData(formulario);
  const { idalumno, ...datosNuevos } = Object.fromEntries(formData.entries());

  try {

```

```

        await updatealumnoCollection(idalumno, datosNuevos);

bootstrap.Modal.getInstance(document.getElementById("editaralumnoModal")).hide();
;
        window.mostrarAlerta({ tipoToast: "success", mensaje: "¡Alumno
actualizado!" });
        await refrescarTablas();
    } catch (error) {
        console.error("Error al actualizar alumno:", error);
    }
};

/**
 * Procesa la actualización del CURSO
 */
window.actualizarCurso = async function (event) {
    event.preventDefault();
    const formulario = document.querySelector("#formularioCursoEdit");
    const formData = new FormData(formulario);
    const data = Object.fromEntries(formData.entries());

    try {
        // Ajustamos los campos para que coincidan con tu estructura de Firebase
        const camposUpdate = {
            curso: data.curso,
            ubicacion: data.ubicacion,
            capacidad: data.capacidad,
            obs: data.obsCurso // En tu DB es 'obs' pero el input suele ser
'obsCurso'
        };
        await updateCursoCollection(data.idCurso, camposUpdate);

bootstrap.Modal.getInstance(document.getElementById("editarCursoModal")).hide();
        window.mostrarAlerta({ tipoToast: "success", mensaje: "¡Curso
actualizado!" });
        await refrescarTablas();
    } catch (error) {
        console.error("Error al actualizar curso:", error);
    }
};

/**
 * Lógica de borrado de ALUMNO (Físico o Lógico según huella)
 */
async function eliminaralumno(id) {
    try {
        const alumnoSnapshot = await getalumnoCollection(id);
        if (!alumnoSnapshot.exists()) return;

        const alumno = alumnoSnapshot.val();

        // Si no tiene huella asignada (-1), borrado físico directo
        if (alumno.huellaId === "-1" || alumno.huellaId === -1) {
            await deletealumnoCollection(id);

```

```

        window.mostrarAlerta({ tipoToast: "success", mensaje: "Alumno
eliminado correctamente" });
    } else {
        // Si tiene huella, borrado lógico para que el ESP32 lo procese
        await updatealumnoCollection(id, { borrar: "Si" });
        window.mostrarAlerta({
            tipoToast: "success",
            mensaje: "Enviando orden de borrado al sensor ESP"
        });
    }
    await refrescarTablas();
} catch (error) {
    console.error("Error en eliminaralumno:", error);
    window.mostrarAlerta({ tipoToast: "error", mensaje: "Error al eliminar"
});
}
}

/**
 * Lógica de borrado de CURSO
 */
async function eliminarCurso(id) {
    try {
        await deleteCursoCollection(id);
        // Eliminación visual inmediata de la fila
        const fila = document.getElementById(id);
        if (fila) fila.remove();

        window.mostrarAlerta({ tipoToast: "success", mensaje: "Curso eliminado
correctamente" });
        await refrescarTablas();
    } catch (error) {
        console.error("Error al borrar el Curso:", error);
        window.mostrarAlerta({ tipoToast: "error", mensaje: "Error al eliminar
el Curso" });
    }
}

```